ISLS v1.0.0

Intelligent Semantic Ledger Substrate

Pure Rust Implementation Specification for Autonomous Build

This document is a **complete, closed, gap-free** implementation specification translating the ISLS Canonical Master Specification (38 pp.) into a buildable Rust workspace. All eight Open Issues (OI-01 through OI-08) are resolved herein. MCCE v1.0.0 is assimilated as Layer L1, ECLS v1.0.0 as Layer L2. An autonomous agent (Claude Code) receiving this document SHALL produce a compiling, test-passing Rust workspace without further clarification.

Author: Sebastian Klemm

Date: 12 March 2026

Normative Source: ISLS Master Spec, Consolidation Report, Handoff Block

Open Issues Resolved: OI-01 through OI-08 (all)

Target: Rust 2021 edition, stable toolchain, no `unsafe`

Conformance Level: High-Rigor (all MUST, SHOULD, and MAY requirements implemented).

Contents

1 Open Issue Resolutions **2**

2 Workspace Structure **3**
- 2.1 External Dependencies (Pinned) . . . 4

3 Crate C1: `isls-types` **4**
- 3.1 Temporal Types . . . 4
- 3.2 5D State Types . . . 4
- 3.3 Carrier Geometry Types . . . 4
- 3.4 Graph and Edge Types . . . 5
- 3.5 Observation, Constraint, Crystal Types . . . 5
- 3.6 Evidence, Proof, Gate Types . . . 6
- 3.7 Configuration . . . 7
- 3.8 Canonical Hashing (OI-01) . . . 7

4 Crate C2: `isls-observe` (Layer L0) **8**

5 Crate C3: `isls-persist` (Layer L1 / MCCE) **8**

6 Crate C4: `isls-extract` (Layer L2 / ECLS) **8**

7 Crate C5: `isls-consensus` (Layer L3) **9**
- 7.1 Normalization Functions (ISLS Sec 10) . . . 9
- 7.2 Metric Computation (All 11 Metrics) . . . 9
- 7.3 PoR State Machine (ISLS Def 15.4) . . . 10
- 7.4 Dual Consensus (ISLS Req 15.1) . . . 10

8 Crate C6: `isls-carrier` **11**

9 Crate C7: `isls-archive` **11**

10 Crate C8: `isls-morph` (Layer L4) **12**

11 Crate C9: `isls-engine` (Orchestrator) **12**
- 11.1 State Machine (ISLS Def 21.1) . . . 12
- 11.2 Global State (ISLS Def 17.1) . . . 12
- 11.3 Macro-Step (ISLS Algo 2) . . . 13
- 11.4 Intrinsic Dynamics (OI-08 Resolved) . . . 14

12 Invariant Enforcement **14**

13 Acceptance Test Suite (AT-01 through AT-20) **15**

14 Build and Run Instructions **16**

1 Open Issue Resolutions

Every open issue from the ISLS Open Issues Register is resolved below. These decisions are **normative** for this implementation.

OI-01 RESOLVED: Canonical Serialization Grammar

Decision: RFC 8785 JSON Canonicalization Scheme (JCS).
Rationale: Human-readable, deterministic, widely implemented, sufficient performance for single-operator system. The crate `serde_jcs` (or manual implementation of RFC 8785 rules atop `serde_json`) SHALL be used.
Rules: Lexicographic key sort by UTF-8 bytes. Numbers: IEEE 754 double-precision serialized per ECMAScript `Number` rules (no trailing zeros, no + sign). No whitespace. No BOM. UTF-8 only.
Hash: SHA-256 over the canonical JCS byte string. The result is a 32-byte `Hash256` = `[u8; 32]`.

OI-02 RESOLVED: Operator Alphabet and Typing

Decision: Define a core trait `Operator` with associated types, plus 8 reference implementations.
Type system:

```rust
pub trait Operator: Send + Sync {
    fn id(&self) -> &str;
    fn version(&self) -> &str;
    fn evaluate(&self, input: &FiveDState, params: &ParamMap) -> f64; // -> [0,1]
    fn is_deterministic(&self) -> bool { true }
}
```

Reference operators: `BandOp`, `RatioOp`, `CorrelationOp`, `GrangerOp`, `SpectralOp`, `TopologicalOp`, `PhaseOp`, `ContractionOp`. Each has fixed type signature, parameter schema, and at least 3 test vectors.

OI-03 RESOLVED: Topological Signature Schema

Decision: Fixed schema with 5 fields.

```rust
pub struct TopologySignature {
    pub betti_0: u64,        // connected components
    pub betti_1: u64,        // 1-cycles (loops)
    pub betti_2: u64,        // 2-voids
    pub spectral_gap: f64,   // lambda_2 - lambda_1 of graph Laplacian
    pub euler_char: i64,     // V - E + F
}
```

Computation: Betti numbers via incremental Vietoris–Rips at scale ε from config. Spectral gap via eigendecomposition of the graph Laplacian (use `nalgebra` for small graphs, iterative Lanczos for large). Euler characteristic from the simplicial complex.

OI-04 RESOLVED: Morphogenic Admissibility Algebra

Decision: Five mutation primitives with explicit preconditions.

Primitive	Precondition	Postcondition
`NodeSplit(v)`	$\psi_v > \theta_{\text{split}}$	Two children inherit edges; parent deactivated
`NodeMerge(u,v)`	$d(u,v) < \epsilon_{\text{merge}}$	Single node; edges unioned
`EdgeRetype(e,t)`	Edge e exists	Type changed; annotation preserved
`SubgraphReplicate(S)`	$\lvert S\rvert \leq k_{\text{rep}}$	Copy with fresh IDs; no shared state
`SubgraphPrune(S)`	All edges in S dormant $> T_{\text{prune}}$	Nodes deactivated (not deleted)

Invariant: No mutation may alter any committed crystal's evidence chain, digest, or provenance.

OI-05 RESOLVED: Proof-Object Verifier Algorithm

Decision: Deterministic function `verify_crystal` returns `VerifyResult`.

```rust
pub fn verify_crystal(crystal: &SemanticCrystal, archive: &Archive) -> VerifyResult {
    // 1. Hash-chain integrity
    for (i, entry) in crystal.evidence_chain.iter().enumerate() {
        let recomputed = sha256_jcs(&entry.content);
        if recomputed != entry.digest { return VerifyResult::Fail("hash mismatch"); }
```

```
        if i > 0 && entry.prev != Some(crystal.evidence_chain[i-1].digest) {
            return VerifyResult::Fail("chain break");
        }
    }
    // 2. Operator-version match
    for (name, ver) in &crystal.commit_proof.operator_stack {
        if archive.pinned_version(name) != Some(ver) {
            return VerifyResult::Fail("operator drift");
        }
    }
    // 3. Gate satisfaction: all 8 gates >= threshold
    // 4. Dual consensus: primal >= theta AND dual >= theta AND mci >= eta
    // 5. PoR trace: monotonic timestamps, T_min/T_stable/T_latch satisfied
    // 6. Content address: crystal.crystal_id == sha256_jcs(canonical(crystal))
    VerifyResult::Accept
}
```

OI-06 RESOLVED: Temperature Calibration

Decision: $T = c_T \cdot \sigma_{\text{realized}}(W)$ where σ_{realized} is the realized standard deviation of the resonance field over window W, and $c_T > 0$ is a configurable scaling constant (default: $c_T = 5.0$).
Fallback: If no domain data is available, $T = T_{\text{default}} = 1.0$.
Regime mapping: $T < 0.5$: calm. $0.5 \leq T < 2.0$: normal. $T \geq 2.0$: volatile.

OI-07 RESOLVED: Radial Coupling Function

Decision: $g(r) = \exp(-\mu_r \cdot r^2)$ with $\mu_r > 0$ configurable (default: $\mu_r = 1.0$).
Verification: $g(0) = 1$, $g(r) \to 0$ as $r \to \infty$, g is smooth and monotone decreasing for $r \geq 0$. Mandorla coupling $\kappa(t) = \exp(-\lambda \cdot \delta\varphi(t)) \cdot \exp(-\mu_r \cdot r(t)^2) \in [0, 1]$.

OI-08 RESOLVED: Intrinsic Flow Field

Decision: Gradient flow on resonance potential.

$$\Phi(\mathbf{H}) = -\nabla V(\mathbf{H}), \quad V(\mathbf{H}) = \tfrac{1}{2}\|\mathbf{H} - \mathbf{H}^*\|^2 + \sum_i \lambda_i \cdot g_i(\mathbf{H})$$

where $\mathbf{H}^*$ is the current attractor centroid (centroid of the top-k resonant points in the HDAG), g_i are active constraint penalty terms, and $\lambda_i > 0$ are Lagrange multipliers from the constraint program.
Discretization: Forward Euler with step Δt_2: $\mathbf{H}_{n+1} = \mathbf{H}_n + \Delta t_2 \cdot \Phi(\mathbf{H}_n)$.
Dissipation: Add damping $-\gamma\mathbf{H}$ with $\gamma > 0$ (default: $\gamma = 0.01$).

2 Workspace Structure

```
isls/
  Cargo.toml                    # [workspace] members = ["crates/*"]
  crates/
    isls-types/                 # C1: canonical data model, serialization, hashing
    isls-observe/               # C2: observation adapters (L0)
    isls-persist/               # C3: persistent HDAG engine (L1 = MCCE assimilated)
    isls-extract/               # C4: constraint extraction (L2 = ECLS assimilated)
    isls-consensus/             # C5: consensus, PoR gate, proof engine (L3)
    isls-carrier/               # C6: tubus, helix, mandorla, phase-ladder
    isls-archive/               # C7: evidence chains, replay, verification
    isls-morph/                 # C8: morphogenic controller (L4)
    isls-engine/                # C9: state machine, orchestrator, acceptance tests
```

Dependency rule: No cycles. `isls-types` is the leaf. `isls-engine` depends on all.

2.1 External Dependencies (Pinned)

Crate	Version	Purpose
`serde`, `serde_json`	1.x	Serialization
`sha2`	0.10	SHA-256 (OI-01)
`petgraph`	0.6	Directed graph
`ordered-float`	4.x	Deterministic float ordering
`nalgebra`	0.33	Linear algebra (eigenvalues for OI-03)
`chrono`	0.4	Timestamps
`thiserror`	2.x	Error types
`tracing`	0.1	Structured logging
`tokio`	1.x	Async runtime (`isls-engine` only)

Prohibitions: No `unsafe`. No `rand` (seeds via `RunDescriptor`). No `std::collections::HashMap` in deterministic paths (use `BTreeMap`).

3 Crate C1: `isls-types`

All shared types, canonical serialization, and content-addressing. Zero internal dependencies.

3.1 Temporal Types

```rust
/// Pre-temporal null-center. Stateless. Cannot hold data. (Axiom 3.7, Inv I13)
pub struct NullCenter; // unit struct; no fields permitted

/// Intrinsic time (continuous internal evolution)
pub type IntrinsicTime = ordered_float::OrderedFloat<f64>; // t2 >= 0

/// Extrinsic time / commit index (discrete irreversible sequence)
pub type CommitIndex = u64; // k in N

/// Tick size for intrinsic discretization
pub type TickSize = ordered_float::OrderedFloat<f64>; // Delta_t2 > 0
```

3.2 5D State Types

```rust
/// Primitive 5D state (Level A): (potential, density, frequency, connectivity, causality)
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct FiveDState {
    pub p: f64,     // potential
    pub rho: f64,   // density
    pub omega: f64, // frequency
    pub chi: f64,   // connectivity
    pub eta: f64,   // causality
}

impl FiveDState {
    pub fn as_array(&self) -> [f64; 5] { [self.p, self.rho, self.omega, self.chi, self.eta] }
    pub fn norm_sq(&self) -> f64 { self.as_array().iter().map(|x| x*x).sum() }
    pub fn distance(&self, other: &Self) -> f64 {
        self.as_array().iter().zip(other.as_array().iter())
            .map(|(a,b)| (a-b)*(a-b)).sum::<f64>().sqrt()
    }
}

/// Tripolar state: 3D projection of 5D (coherence amplitude, population, phase frequency)
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct TripolarState {
    pub psi: f64,   // coherence amplitude in [0,1]
    pub rho: f64,   // population/density in [0,1]
    pub omega: f64, // instantaneous phase frequency
}
```

3.3 Carrier Geometry Types

```rust
/// Tubus coordinate: (tau, phi, r) in I x S^1 x R>=0
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct TubusCoord {
```

```rust
    pub tau: f64,    // directed carrier coordinate (forward progression)
    pub phi: f64,    // cyclic helix phase in [0, 2*pi)
    pub r: f64,      // radial distance >= 0
}

/// Mandorla state: interference of helix pair
#[derive(Clone, Debug, PartialEq, Serialize, Deserialize)]
pub struct MandorlaState {
    pub tau: f64,         // shared carrier coordinate
    pub r: f64,           // shared radius
    pub delta_phi: f64,   // phase differential
    pub kappa: f64,       // coupling strength in [0,1]
}

/// Carrier instance (one entry in the phase-ladder)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct CarrierInstance {
    pub helix_a: TubusCoord,
    pub helix_b: TubusCoord,
    pub mandorla: MandorlaState,
    pub resonance: f64,
    pub offset: f64,  // phase offset delta_i in [0, 2*pi)
}

/// Phase-ladder: ordered set of carrier instances
pub type PhaseLadder = Vec<CarrierInstance>;
```

3.4 Graph and Edge Types

```rust
pub type VertexId = u64;
pub type Hash256 = [u8; 32];

/// Edge annotation (MCCE 7-tuple, ISLS Def 4.6)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct EdgeAnnotation {
    pub correlation: f64,      // rho in [-1,1]
    pub granger: f64,          // tau in R (Granger causality)
    pub coherence: f64,        // kappa in [0,1] (spectral)
    pub weight: f64,           // w >= 0 (resonance weight)
    pub birth_time: f64,       // first observation timestamp
    pub last_update: f64,      // last significant update
    pub active_windows: u64,   // n_e count
}

/// Topological signature (OI-03 resolved)
#[derive(Clone, Debug, Default, Serialize, Deserialize)]
pub struct TopologySignature {
    pub betti_0: u64,
    pub betti_1: u64,
    pub betti_2: u64,
    pub spectral_gap: f64,
    pub euler_char: i64,
}
```

3.5 Observation, Constraint, Crystal Types

```rust
/// Canonical observation record (L0 output)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct Observation {
    pub timestamp: f64,
    pub source_id: String,
    pub provenance: ProvenanceEnvelope,
    pub payload: Vec<u8>,
    pub context: MeasurementContext,
    pub digest: Hash256,
    pub schema_version: String,
}

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct ProvenanceEnvelope { pub origin: String, pub chain: Vec<String>, pub sig: Option<Vec<u8>>
     }
```

```rust
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct MeasurementContext { pub tags: BTreeMap<String, String> }

/// Constraint template family (OI-02)
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq, Eq, Hash)]
pub enum ConstraintTemplate { Band, Ratio, Correlation, Granger, Spectral, Topological, Phase,
    Contraction }

/// Constraint candidate (ISLS Def 14.1)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct ConstraintCandidate {
    pub id: Hash256,
    pub template: ConstraintTemplate,
    pub parameters: BTreeMap<String, f64>,
    pub coverage: f64,      // alpha in [0,1]
    pub threshold: f64,     // theta
    pub formation_energy: f64, // Delta G (negative = spontaneous)
    pub bond_strength: u64,    // windows active
    pub activation_energy: f64, // E_a
}

/// Constraint program = ordered sequence of candidates
pub type ConstraintProgram = Vec<ConstraintCandidate>;

/// Semantic crystal (ISLS Def 4.8, enriched)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct SemanticCrystal {
    pub crystal_id: Hash256,
    pub region: Vec<VertexId>,          // condensed subgraph vertices
    pub constraint_program: ConstraintProgram,
    pub stability_score: f64,           // K*_k in [0,1]
    pub topology_signature: TopologySignature,
    pub betti_numbers: Vec<u64>,
    pub evidence_chain: EvidenceChain,
    pub commit_proof: CommitProof,
    pub operator_versions: BTreeMap<String, String>,
    pub created_at: CommitIndex,
    pub free_energy: f64,
    pub carrier_instance_idx: usize,
}
```

3.6 Evidence, Proof, Gate Types

```rust
/// Evidence entry in an evidence chain
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct EvidenceEntry {
    pub digest: Hash256,
    pub content: Vec<u8>,
    pub provenance: ProvenanceEnvelope,
    pub prev: Option<Hash256>,
}
pub type EvidenceChain = Vec<EvidenceEntry>;

/// Gate snapshot: all 8 normalized metrics + Kairos conjunction
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct GateSnapshot {
    pub d: f64, pub q: f64, pub r: f64, pub g: f64,
    pub j: f64, pub p: f64, pub n: f64, pub k: f64,
    pub kairos: bool,
}

/// PoR finite-state machine trace
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct PoRTrace {
    pub search_enter: f64,
    pub lock_enter: Option<f64>,
    pub verify_enter: Option<f64>,
    pub commit_enter: Option<f64>,
}

/// Consensus result (primal + dual scores + MCI)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct ConsensusResult {
```

```rust
    pub primal_score: f64,
    pub dual_score: f64,
    pub mci: f64,
    pub threshold: f64,
}

/// Commit proof (ISLS Sec 25.2)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct CommitProof {
    pub evidence_digests: Vec<Hash256>,
    pub operator_stack: Vec<(String, String)>,  // (name, version)
    pub gate_values: GateSnapshot,
    pub structural_result: bool,
    pub consensus_result: ConsensusResult,
    pub por_trace: PoRTrace,
    pub carrier_id: usize,
    pub carrier_offset: f64,
}

/// Run descriptor for replay determinism (ISLS Def 28.1)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct RunDescriptor {
    pub config: Config,
    pub operator_versions: BTreeMap<String, String>,
    pub initial_state_digest: Hash256,
    pub seed: Option<u64>,
}
```

3.7 Configuration

```rust
/// Master configuration (ISLS Sec 33, all OIs resolved)
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct Config {
    pub temporal: TemporalConfig,
    pub carrier: CarrierConfig,
    pub observation: ObservationConfig,
    pub persistence: PersistenceConfig,
    pub extraction: ExtractionConfig,
    pub consensus: ConsensusConfig,
    pub adaptation: AdaptationConfig,
    pub thresholds: ThresholdConfig,
    pub normalization: NormalizationConfig,
    pub archive: ArchiveConfig,
}

#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct ThresholdConfig {
    pub d: f64, pub q: f64, pub r: f64, pub g: f64,
    pub j: f64, pub p: f64, pub n: f64, pub k: f64,
    pub f_friction: f64, pub s_shock: f64, pub l_migration: f64,
}
// (remaining sub-configs follow same pattern with fields from ISLS Sec 33)
```

3.8 Canonical Hashing (OI-01)

```rust
/// Canonical JCS serialization + SHA-256 content addressing
pub fn canonical_bytes<T: Serialize>(value: &T) -> Vec<u8> {
    // RFC 8785: sorted keys, no whitespace, ECMAScript number format
    serde_jcs::to_vec(value).expect("JCS serialization must not fail for canonical types")
}

pub fn content_address<T: Serialize>(value: &T) -> Hash256 {
    use sha2::{Sha256, Digest};
    let bytes = canonical_bytes(value);
    let mut hasher = Sha256::new();
    hasher.update(&bytes);
    hasher.finalize().into()
}
```

4 Crate C2: isls-observe (Layer L0)

```rust
/// Observation canonicalization map: Gamma_obs (ISLS Def 12.1)
pub trait ObservationAdapter: Send + Sync {
    fn source_id(&self) -> &str;
    fn canonicalize(&self, raw: &[u8], context: &MeasurementContext) -> Result<Observation>;
}

/// Idempotent ingestion: same raw input -> same digest (Req 12.2)
pub fn ingest(adapter: &dyn ObservationAdapter, raw: &[u8], ctx: &MeasurementContext)
    -> Result<Observation>
{
    let obs = adapter.canonicalize(raw, ctx)?;
    // Verify: re-canonicalize produces identical digest
    debug_assert_eq!(content_address(&obs.payload), obs.digest);
    Ok(obs)
}
```

Observation classes: `StateObs`, `RelationObs`, `EventObs`, `PhaseObs`, `ExogenousObs`.

5 Crate C3: isls-persist (Layer L1 / MCCE)

This crate assimilates MCCE v1.0.0. The "Mycelial HDAG" becomes the ISLS persistent structural graph.

```rust
/// Persistent graph (ISLS Def 4.5, MCCE HDAG)
pub struct PersistentGraph {
    graph: petgraph::Graph<VertexData, EdgeAnnotation, petgraph::Directed>,
    id_map: BTreeMap<VertexId, petgraph::graph::NodeIndex>,
    tensor: BTreeMap<VertexId, TensorArchive>,  // Lambda: V -> R^{5xTxK}
    embedding: BTreeMap<VertexId, FiveDState>,    // Phi: V -> H5
    hot: HotTier,
    warm: WarmTier,
    cold: ColdTier,
    commit_index: CommitIndex,
}

/// Persistence transition: G_{k+1} = T_persist(G_k, D_obs, theta) (ISLS Sec 13.2)
impl PersistentGraph {
    pub fn apply_observations(&mut self, obs_batch: &[Observation], config: &PersistenceConfig)
        -> Result<()>
    {
        for obs in obs_batch {
            // 1. Upsert vertices referenced in observation
            // 2. Update edge annotations (rolling correlation, Granger, coherence)
            // 3. Append to tensor archive
            // 4. Update embeddings
            // 5. Decay dormant edges: w *= exp(-lambda_decay * delta_t)
            // 6. Tier migration (hot -> warm -> cold) based on retention policy
        }
        self.commit_index += 1;
        Ok(())
    }

    /// Append-only invariant: never delete edges or vertex history (Inv I1)
    pub fn deactivate_vertex(&mut self, id: VertexId) { /* mark inactive, don't remove */ }
}
```

Storage tiers: `HotTier` (7 days, in-memory), `WarmTier` (90 days, compressed on-disk), `ColdTier` (indefinite, append-only files).

6 Crate C4: isls-extract (Layer L2 / ECLS)

This crate assimilates ECLS v1.0.0. Constraint discovery operates **read-only** on the persistent graph (Inv I5).

```rust
/// Inverse Weaving: recover emergent constraint program (ISLS Algo 1)
pub fn inverse_weave(
    graph: &PersistentGraph,
    window: &TimeWindow,
    library: &[Box<dyn Operator>],
```

```rust
    config: &ExtractionConfig,
) -> (ConstraintProgram, Vec<VertexId>) // (infogenome, condensed region K*)
{
    let point_cloud = graph.point_cloud_over(window);
    let mut active: Vec<ConstraintCandidate> = library.iter()
        .filter_map(|op| scan_constraint(op, &point_cloud, config))
        .filter(|c| c.coverage >= config.alpha_min)
        .collect();

    let mut program = ConstraintProgram::new();
    let mut region = point_cloud.clone();

    while !active.is_empty() && variance(&region) > config.convergence_tau {
        // Select max info-gain per risk: q(g) = (H(K) - H(K cap g)) / (risk(g) + eps)
        let best = select_best_constraint(&active, &region, config);
        region.retain(|v| best.evaluate(v) >= best.threshold);
        let contraction = region.len() as f64 / point_cloud.len() as f64;
        program.push(best.clone());
        active.retain(|c| c.id != best.id);
        if contraction > config.kappa_max { break; }
    }
    (program, region.into_vertex_ids())
}

/// Lattice free energy: F(L) = -sum(q(g_i)) + T * H(K*) (ISLS Def 14.4, OI-06)
pub fn lattice_free_energy(program: &ConstraintProgram, region: &[FiveDState],
    temperature: f64) -> f64
{
    let info_gain: f64 = program.iter().map(|c| -c.formation_energy).sum();
    let entropy = region_entropy(region);
    -info_gain + temperature * entropy
}
```

7 Crate C5: `isls-consensus` (Layer L3)

7.1 Normalization Functions (ISLS Sec 10)

```rust
/// Saturation normalization: N(u; mu) = u / (u + mu) (Def 10.1)
pub fn norm_saturate(u: f64, mu: f64) -> f64 { u / (u + mu) }

/// Exponential normalization: N_exp(d; lambda) = exp(-lambda * d) (Def 10.2)
pub fn norm_exp(d: f64, lambda: f64) -> f64 { (-lambda * d).exp() }
```

7.2 Metric Computation (All 11 Metrics)

```rust
pub struct MetricSet {
    pub d_deformation: f64,    // Def 10.3: N(D_raw; mu_D)
    pub q_coherence: f64,      // Def 10.4: N(Q_raw; mu_Q)
    pub r_resonance: f64,      // Def 10.5: exp(-d_R(H, s_ref))
    pub g_readiness: f64,      // Def 10.6: gamma_D*D + gamma_Q*Q + gamma_R*R
    pub j_doublekick: f64,     // Def 10.7: N(J_raw; mu_J)
    pub p_projection: f64,     // Def 10.8: exp(-diam(P) * lambda_P)
    pub n_seam: f64,           // Def 10.9: exp(-d_seam(L,R))
    pub k_crystal: f64,        // Def 10.10: lambda_C*C + lambda_E*E
    pub f_friction: f64,       // Def 10.11: N(F_raw; mu_F)
    pub s_shock: f64,          // Def 10.12: N(S_raw; mu_S)
    pub l_migration: f64,      // from carrier readiness
}

impl MetricSet {
    pub fn gate_snapshot(&self, thresholds: &ThresholdConfig) -> GateSnapshot {
        GateSnapshot {
            d: self.d_deformation, q: self.q_coherence, r: self.r_resonance,
            g: self.g_readiness, j: self.j_doublekick, p: self.p_projection,
            n: self.n_seam, k: self.k_crystal,
            kairos: self.d_deformation >= thresholds.d && self.q_coherence >= thresholds.q
                && self.r_resonance >= thresholds.r && self.g_readiness >= thresholds.g
                && self.j_doublekick >= thresholds.j && self.p_projection >= thresholds.p
                && self.n_seam >= thresholds.n && self.k_crystal >= thresholds.k,
        }
    }
```

```
}
```

7.3 PoR State Machine (ISLS Def 15.4)

```
#[derive(Clone, Debug, PartialEq)]
pub enum PoRState { Search, Lock, Verify, Commit }

pub struct PoRFsm {
    state: PoRState,
    stability_history: Vec<f64>,
    lock_tick: Option<f64>,
    verify_tick: Option<f64>,
    trace: PoRTrace,
}

impl PoRFsm {
    pub fn step(&mut self, kappa: f64, t2: f64, config: &ConsensusConfig) -> PoRState {
        match self.state {
            PoRState::Search => {
                if kappa >= config.por_kappa_bar {
                    self.stability_history.push(kappa);
                    if self.stability_history.len() >= config.por_t_min {
                        self.state = PoRState::Lock;
                        self.lock_tick = Some(t2);
                        self.trace.lock_enter = Some(t2);
                    }
                } else { self.stability_history.clear(); }
            }
            PoRState::Lock => {
                let delta = (kappa - self.stability_history.last().unwrap_or(&0.0)).abs();
                if delta <= config.por_epsilon {
                    self.stability_history.push(kappa);
                    if self.stability_history.len() >= config.por_t_min + config.por_t_stable {
                        self.state = PoRState::Verify;
                        self.verify_tick = Some(t2);
                        self.trace.verify_enter = Some(t2);
                    }
                } else { self.reset(t2); }
            }
            PoRState::Verify => { /* check policy constraints, latch time */ }
            PoRState::Commit => { /* terminal until reset */ }
        }
        self.state.clone()
    }
    fn reset(&mut self, t2: f64) {
        self.state = PoRState::Search;
        self.stability_history.clear();
        self.trace = PoRTrace { search_enter: t2, ..Default::default() };
    }
}
```

7.4 Dual Consensus (ISLS Req 15.1)

```
/// Run primal and dual operator paths; check MCI (OI-04/OI-05)
pub fn dual_consensus(
    precursor: &CrystalPrecursor,
    primal_ops: &[&dyn CascadeOperator],    // DK -> SW -> PI -> WT
    dual_ops: &[&dyn CascadeOperator],      // PI -> WT -> DK -> SW
    config: &ConsensusConfig,
) -> ConsensusResult {
    let primal_state = run_cascade(precursor, primal_ops);
    let dual_state = run_cascade(precursor, dual_ops);
    let mci = 1.0 - primal_state.distance(&dual_state);
    ConsensusResult {
        primal_score: primal_state.stability_score(),
        dual_score: dual_state.stability_score(),
        mci,
        threshold: config.consensus_threshold,
    }
}
```

8 Crate C6: isls-carrier

```rust
/// Helix pair with pi-phase coupling (ISLS Def 7.2, Inv I15)
pub fn helix_pair(tau: f64, phi: f64, r: f64) -> (TubusCoord, TubusCoord) {
    let alpha = TubusCoord { tau, phi, r };
    let beta = TubusCoord { tau, phi: (phi + std::f64::consts::PI) % (2.0 * std::f64::consts::PI), r
        };
    (alpha, beta)
}

/// Mandorla formation (ISLS Def 7.3, OI-07 resolved)
pub fn mandorla(alpha: &TubusCoord, beta: &TubusCoord, lambda: f64, mu_r: f64) -> MandorlaState {
    let delta_phi = (alpha.phi - beta.phi).abs().min(
        2.0 * std::f64::consts::PI - (alpha.phi - beta.phi).abs());
    let kappa = (-lambda * delta_phi).exp() * (-mu_r * alpha.r * alpha.r).exp();
    MandorlaState { tau: alpha.tau, r: alpha.r, delta_phi, kappa }
}

/// Carrier migration admissibility (ISLS Def 8.3)
pub fn migration_admissible(metrics: &MetricSet, target: &CarrierInstance,
    config: &CarrierConfig) -> bool
{
    let friction_or_shock = metrics.f_friction >= config.thresholds.f_friction
        || metrics.s_shock >= config.thresholds.s_shock;
    let readiness = config.lambda_q * target.resonance
        + config.lambda_r * 0.5 /* target R */
        + config.lambda_m * target.mandorla.kappa;
    friction_or_shock && readiness >= config.thresholds.l_migration
}
```

9 Crate C7: isls-archive

```rust
/// Build evidence chain from observation sequence
pub fn build_evidence_chain(entries: &[Vec<u8>]) -> EvidenceChain {
    let mut chain = Vec::with_capacity(entries.len());
    for (i, content) in entries.iter().enumerate() {
        let digest = content_address_raw(content);
        let prev = if i > 0 { Some(chain[i-1].digest) } else { None };
        chain.push(EvidenceEntry {
            digest, content: content.clone(),
            provenance: ProvenanceEnvelope::default(), prev,
        });
    }
    chain
}

/// Verify crystal integrity (OI-05 resolved)
pub fn verify_crystal(crystal: &SemanticCrystal, pinned: &BTreeMap<String,String>)
    -> Result<(), VerifyError>
{
    // 1. Content address
    let recomputed_id = content_address(crystal);
    if recomputed_id != crystal.crystal_id { return Err(VerifyError::ContentAddress); }

    // 2. Hash-chain integrity
    for (i, entry) in crystal.evidence_chain.iter().enumerate() {
        if content_address_raw(&entry.content) != entry.digest {
            return Err(VerifyError::HashChain(i));
        }
        if i > 0 && entry.prev != Some(crystal.evidence_chain[i-1].digest) {
            return Err(VerifyError::ChainBreak(i));
        }
    }

    // 3. Operator versions
    for (name, ver) in &crystal.commit_proof.operator_stack {
        if pinned.get(name) != Some(ver) { return Err(VerifyError::OperatorDrift(name.clone())); }
    }

    // 4. Gate satisfaction (all metrics >= thresholds checked in GateSnapshot.kairos)
```

```
    if !crystal.commit_proof.gate_values.kairos { return Err(VerifyError::GateFail); }

    // 5. Dual consensus
    let cr = &crystal.commit_proof.consensus_result;
    if cr.primal_score < cr.threshold || cr.dual_score < cr.threshold || cr.mci < 0.8 {
        return Err(VerifyError::ConsensusFail);
    }

    // 6. PoR trace monotonicity
    // (verify search <= lock <= verify <= commit timestamps)
    Ok(())
}
```

10 Crate C8: isls-morph (Layer L4)

```
/// Morphogenic mutation primitives (OI-04 resolved)
pub enum MorphMutation {
    NodeSplit { vertex: VertexId },
    NodeMerge { u: VertexId, v: VertexId },
    EdgeRetype { from: VertexId, to: VertexId, new_type: String },
    SubgraphReplicate { vertices: Vec<VertexId> },
    SubgraphPrune { vertices: Vec<VertexId> },
}

/// Morphogenic update: mu_{k+1} = A_morph(mu_k, C_<=k, theta) (ISLS Def 16.2)
pub fn morphogenic_update(
    graph: &mut PersistentGraph,
    morph_state: &mut MorphState,
    crystals: &[SemanticCrystal],
    config: &AdaptationConfig,
) -> Vec<MorphMutation> {
    let mut mutations = Vec::new();
    // Evaluate pressure: H(x,t) = alpha*Phi(x,t) + beta*mu(x,t) (Hyperbion, OI-08)
    let pressure = compute_pressure(graph, morph_state, config);

    for (vid, p) in pressure {
        if p > config.split_threshold {
            mutations.push(MorphMutation::NodeSplit { vertex: vid });
        }
    }
    // Merge nearby nodes below merge distance
    // Prune dormant subgraphs
    // Apply mutations (non-retroactive: Inv I11)
    for m in &mutations { apply_mutation(graph, m, config); }
    mutations
}
```

11 Crate C9: isls-engine (Orchestrator)

11.1 State Machine (ISLS Def 21.1)

```
#[derive(Clone, Debug, PartialEq)]
pub enum EngineState {
    Idle, Observing, Relating, Embedding, MandorlaForming,
    Resonating, KairosPrimed, Migrating, Capturing,
    Projecting, Stitching, Crystallizing, Monolithizing,
    Breaking, Restoring, Committed, Rejected(String),
}
```

11.2 Global State (ISLS Def 17.1)

```
pub struct GlobalState {
    pub graph: PersistentGraph,
    pub candidates: Vec<CrystalPrecursor>,
    pub consensus: ConsensusState,
    pub morph: MorphState,
    pub active_carrier: usize,  // index into phase_ladder
    pub phase_ladder: PhaseLadder,
```

```rust
    pub h5_state: FiveDState,
    pub commit_index: CommitIndex,
    pub engine_state: EngineState,
    pub por_fsm: PoRFsm,
}
```

11.3 Macro-Step (ISLS Algo 2)

```rust
/// T_ISLS(S_k, X_k; theta) = A_morph . C_commit . E_extract . T_persist . Gamma_obs
pub fn macro_step(state: &mut GlobalState, obs_batch: &[Vec<u8>], config: &Config)
    -> Result<Option<SemanticCrystal>>
{
    // L0: Canonicalize observations
    state.engine_state = EngineState::Observing;
    let canonical_obs = obs_batch.iter()
        .map(|raw| ingest(&config.observation.adapter, raw, &MeasurementContext::default()))
        .collect::<Result<Vec<_>>>()?;

    // L1: Update persistent graph (MCCE assimilated)
    state.engine_state = EngineState::Relating;
    state.graph.apply_observations(&canonical_obs, &config.persistence)?;

    // Embedding + Mandorla
    state.engine_state = EngineState::Embedding;
    update_embeddings(&mut state.graph, &state.phase_ladder[state.active_carrier], config);
    state.engine_state = EngineState::MandorlaForming;
    let mandorla = update_mandorla(&state.phase_ladder[state.active_carrier], config);

    // Resonance evaluation
    state.engine_state = EngineState::Resonating;
    let metrics = compute_all_metrics(&state.graph, &mandorla, &state.h5_state, config);

    // Check friction/shock -> migration
    if metrics.f_friction >= config.thresholds.f_friction
        || metrics.s_shock >= config.thresholds.s_shock
    {
        state.engine_state = EngineState::Migrating;
        attempt_carrier_migration(state, &metrics, config);
    }

    // Kairos gate check
    let gate = metrics.gate_snapshot(&config.thresholds);
    if !gate.kairos {
        state.engine_state = EngineState::Rejected("kairos failed".into());
        return Ok(None);
    }
    state.engine_state = EngineState::KairosPrimed;

    // L2: Constraint extraction (ECLS assimilated)
    state.engine_state = EngineState::Capturing;
    let window = TimeWindow::last_n(config.extraction.window_hours);
    let (program, region) = inverse_weave(&state.graph, &window,
        &config.extraction.operators, &config.extraction);

    // Operators: DK + Funnel
    state.engine_state = EngineState::Projecting;
    let condensed = apply_operator_cascade(&state.h5_state, &mandorla, config);

    // Stitching
    state.engine_state = EngineState::Stitching;
    let seam = stitch(&condensed, &region, config);
    if metrics.n_seam < config.thresholds.n {
        state.engine_state = EngineState::Rejected("seam failed".into());
        return Ok(None);
    }

    // L3: Crystallize
    state.engine_state = EngineState::Crystallizing;
    let precursor = CrystalPrecursor { program, region, seam, metrics: metrics.clone() };

    // Dual consensus
    let consensus = dual_consensus(&precursor,
        &config.consensus.primal_ops(), &config.consensus.dual_ops(), &config.consensus);
```

```
    if consensus.primal_score < consensus.threshold
        || consensus.dual_score < consensus.threshold
        || consensus.mci < config.consensus.mirror_consistency_eta
    {
        state.engine_state = EngineState::Rejected("consensus failed".into());
        return Ok(None);
    }

    // Build crystal
    let crystal = build_crystal(precursor, consensus, &gate, state, config);
    state.engine_state = EngineState::Monolithizing;

    // Commit (append to immutable archive)
    state.engine_state = EngineState::Committed;
    state.commit_index += 1;

    // L4: Morphogenic update
    morphogenic_update(&mut state.graph, &mut state.morph, &[crystal.clone()], &config.adaptation);

    // Restore neutrality
    state.engine_state = EngineState::Restoring;
    restore_carrier_neutrality(state, config);
    state.engine_state = EngineState::Idle;

    Ok(Some(crystal))
}
```

11.4 Intrinsic Dynamics (OI-08 Resolved)

```
/// dH/dt2 = Phi(H) = -grad V(H) - gamma*H (gradient flow + damping)
pub fn intrinsic_step(h: &mut FiveDState, attractor: &FiveDState,
    constraints: &ConstraintProgram, dt: f64, gamma: f64)
{
    let mut grad = [0.0f64; 5];
    let ha = h.as_array();
    let aa = attractor.as_array();
    for i in 0..5 {
        grad[i] = (ha[i] - aa[i]) + gamma * ha[i]; // potential gradient + damping
        for c in constraints {
            grad[i] += c.activation_energy * (ha[i] - c.parameters.get("center")
                .copied().unwrap_or(0.0)); // constraint penalty gradient
        }
    }
    // Forward Euler
    *h = FiveDState {
        p: ha[0] - dt * grad[0], rho: ha[1] - dt * grad[1],
        omega: ha[2] - dt * grad[2], chi: ha[3] - dt * grad[3],
        eta: ha[4] - dt * grad[4],
    };
}
```

12 Invariant Enforcement

Invariant Enforcement Map (I1–I20)

Inv	Statement	Enforcement Point
I1	Append-only history	`isls-persist`: no `delete` methods
I2	Canonical serialization	`isls-types`: `canonical_bytes` (JCS)
I3	Provenance completeness	`isls-archive`: `verify_crystal` check 2
I4	Deterministic replay	`isls-engine`: `RunDescriptor` + no `rand`
I5	Read-only extraction	`isls-extract`: takes `&PersistentGraph` (immutable ref)
I6	Atomic embedding	`isls-persist`: single `apply_observations` call
I7	Phase monotonicity	`isls-carrier`: assert `omega_d > 0`
I8	Storage integrity	`isls-persist`: digest-on-read for all tiers
I9	Threshold-governed commit	`isls-consensus`: Kairos gate in `macro_step`
I10	Commit immutability	`isls-archive`: no `&mut` on committed crystals
I11	Morphogenic non-retroactivity	`isls-morph`: past crystal digests unchanged
I12	Resource boundedness	`isls-engine`: configurable limits per cycle
I13	Empty null-center	`isls-types`: `NullCenter` is unit struct
I14	Tri-temporal irreducibility	Separate `IntrinsicTime`, `CommitIndex`, `NullCenter`
I15	Dual helix coupling	`isls-carrier`: `helix_pair` enforces π offset
I16	Mandorla eigenfield	`isls-carrier`: `MandorlaState` $\neq$ `TubusCoord`
I17	No monolith without crystal	`isls-engine`: crystal required before commit
I18	No breaking without Kairos	`isls-engine`: Kairos gate check before action
I19	Traceability	`isls-archive`: `CommitProof` carries full trace
I20	Operator version pinning	`isls-types`: `RunDescriptor`.operator_versions

13 Acceptance Test Suite (AT-01 through AT-20)

All 20 tests from ISLS §29. Implemented as `#[test]` functions in `isls-engine`/tests/.

ID	Name	Rust Test Sketch
AT-01	Idempotent ingestion	Ingest same raw bytes twice; `assert_eq!(obs1.digest, obs2.digest)`
AT-02	Append-only	Decay edges; query historical; assert recoverable
AT-03	Replay determinism	Run `macro_step` twice with same `RunDescriptor`; assert identical crystal digests
AT-04	Read-only extraction	Run `inverse_weave` on `&graph`; assert graph unchanged
AT-05	Constraint convergence	Inject synthetic correlated data; assert crystal emerges within N steps
AT-06	Provenance completeness	For each crystal, `verify_crystal` returns `Ok`
AT-07	Threshold-gated reject	Set one metric below threshold; assert no crystal
AT-08	Positive commit	Present valid candidate above all thresholds; assert crystal appended
AT-09	Storage corruption	Corrupt warm tier; assert re-derivation or explicit error
AT-10	Non-retroactivity	Trigger morph mutation; assert past crystal digests unchanged
AT-11	Operator drift	Change operator version string; assert protocol fault
AT-12	Resource bound	Run 1000 cycles; assert memory/time within declared bounds
AT-13	Dual consensus	Run primal+dual; assert both must agree
AT-14	PoR FSM	Verify search→lock→verify→commit transitions
AT-15	Carrier migration	Inject friction $> \theta_F$; assert clean migration
AT-16	Kairos gate	Suppress one gate; assert no monolith
AT-17	Null-center stateless	Assert `NullCenter` has no fields, no methods that write
AT-18	Tri-temporal ordering	Verify $n_0 \triangleleft t_2 \triangleleft t_1$ in all commit traces
AT-19	Content addressing	Assert $\mathrm{id}(C) = \text{SHA-256}(\mathrm{JCS}(C))$ for all crystals

ID	Name	Rust Test Sketch
AT-20	Symmetry restoration	After break, assert carrier back to neutral before next cycle

14 Build and Run Instructions

```
# Build
cargo build --workspace --release

# Run all acceptance tests
cargo test --workspace -- --test-threads=1  # single-threaded for determinism

# Run specific test
cargo test -p isls-engine at_03_replay_determinism

# Generate reference configuration
cargo run -p isls-engine -- --emit-config > isls_config.json
```

End of specification — ISLS v1.0.0 Rust Implementation Specification

All Open Issues (OI-01 through OI-08) resolved. All 20 Invariants enforced. All 20 Acceptance Tests specified.